DAO

Patrones de diseño
Implementación

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

{ }

```
class DAO:
def aplicar():
return patron
```

Plantilla institucional de clases

Versión 2026

CONTENIDOS

Contenido

01 Patrones

Conceptos clave

- Concepto
- Utilidad

02 Implementación en Python

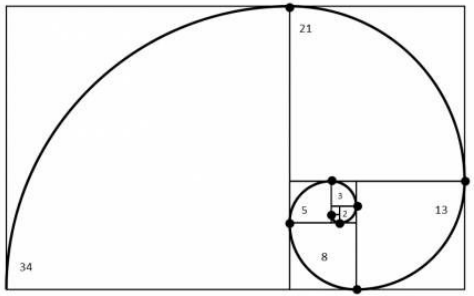
Principales Patrones

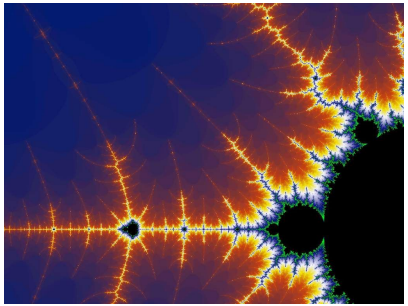
- Singleton
- Observer
- Factory

Patrones: Matemáticos y Gráficos

Matemáticos

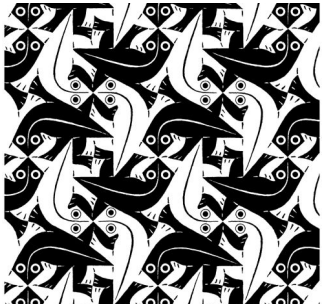
Sucesión de elementos que se construye siguiendo una regla

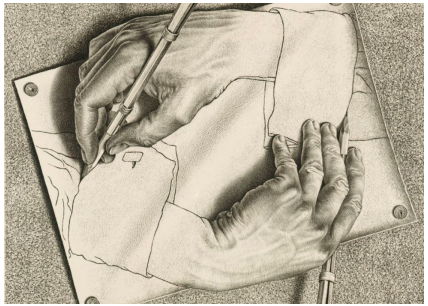




Gráficos

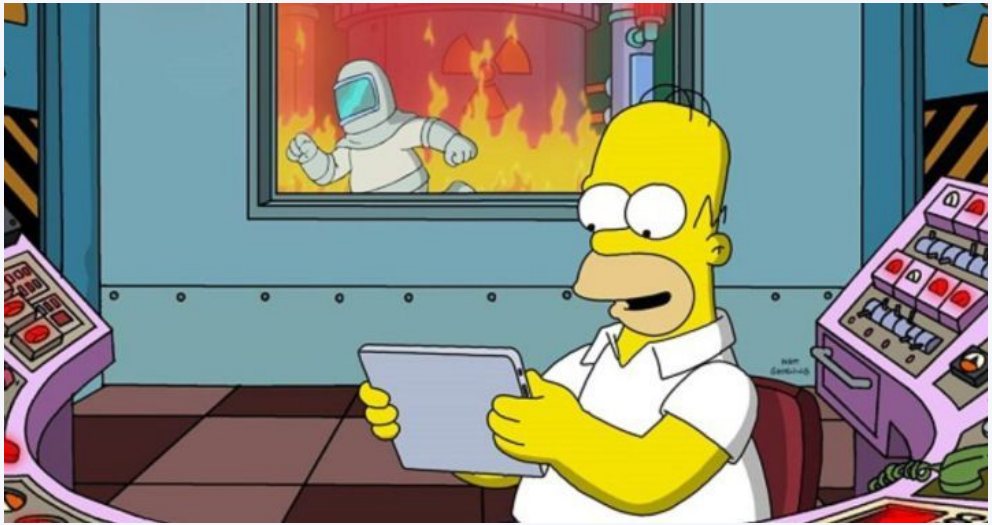
Secuencias de figuras que tienen una cierta regularidad





Patrones de diseño

- Son soluciones generales a problemas comunes en el diseño de software
- Se utilizan para reutilizar “conocimiento”, no necesariamente código



¿Cómo se describen los patrones?



Nombre

Nombre del patrón

Un nombre descriptivo que identifica el patrón. Esto facilita la comunicación entre desarrolladores.

Problema

Contexto y uso

Descripción de la situación o problema donde aplica. Proporciona contexto sobre cuándo y por qué utilizarlo.

Solución

Estructura

Descripción de la solución general, clases, objetos involucrados, relaciones e interacciones para resolver el problema.

Consecuencias

Trade-offs

Implicaciones de aplicar el patrón, ventajas y posibles desventajas para comprender su impacto en el diseño.

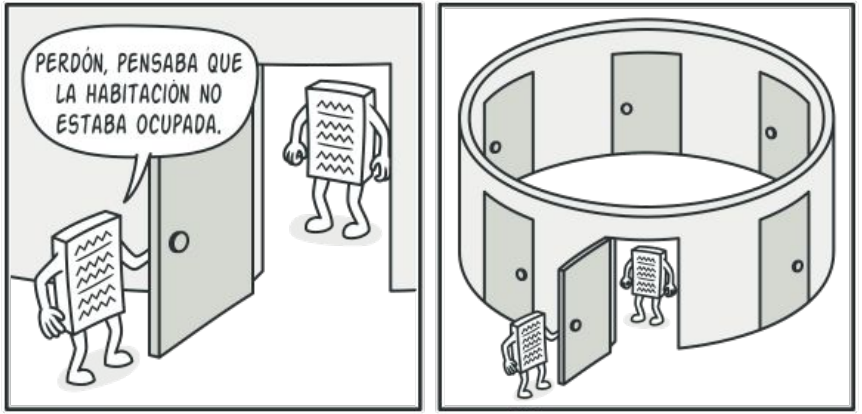
Ejemplos

Implementación

Ejemplos de código o casos de uso prácticos que ilustran cómo implementarlo en un lenguaje específico.

Patrones de Diseño

Singleton



Problema que resuelve:

- Una clase tiene una única instancia
- Esa instancia tiene un único punto de acceso

Se utiliza en:

- Acceso a recursos compartidos
- Centralización de datos y operaciones a nivel de sistema

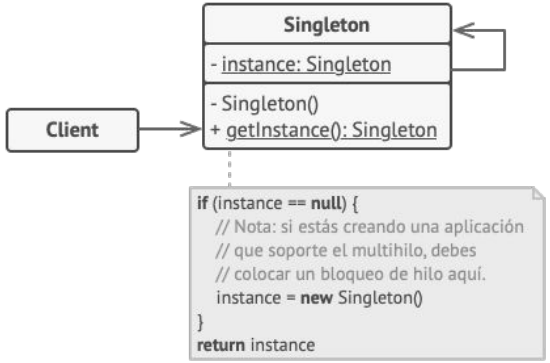
Solución:

- Ocultar el constructor
- Crear un método estático que funcione como constructor
- La primera vez que se llama al método se crea un objeto y se guarda en un campo estático.
- Las siguientes veces simplemente se retorna el objeto almacenado, sin crear uno nuevo.

Singleton: Definición e Implementación

```python
class Singleton:
    # Variable de instancia, para contener el objeto
    _instance = None

    def __new__(cls):
        """El operador __new__ se dispara automáticamente al
        crear un objeto de una clase.
        :cls: Clase que se está creando
        :return: Devuelve un nuevo objeto
        """
        # Si no está asignado _instance
        if cls._instance is None:
            # Pedirle al padre (super), que cree un objeto cls
            # y lo asigne a _instance
            cls._instance = super(Singleton, cls).__new__(cls)
        # Retornar _instance, ya sea si estaba asignado o
        # si se acaba de crear
        return cls._instance
```



Singleton: Ejemplo

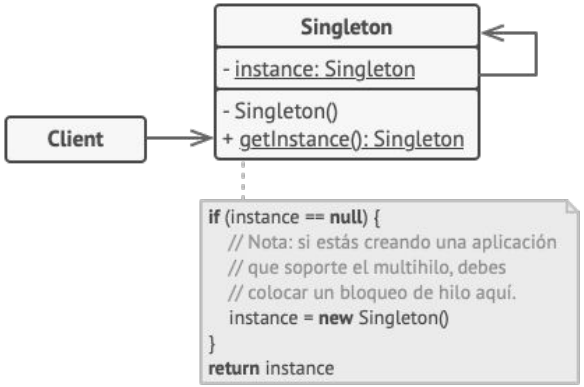
```python
class DBConnection:
    _instance = None
    dbconnection = None

    def __new__ (cls, connection_string : str):
        if not cls._instance:
            cls._instance = super(DBConnection,cls).__new__(cls)
            cls.dbconnection = connection_string
        return cls._instance

db = DBConnection("sqlserver:mcasatti@localhost/DBMAIN")
print (db.dbconnection)

db2 = DBConnection("oracle:martincito@servidor/DATABASE")
print (db.dbconnection)

```



Patrón Factory

Problema que resuelve:

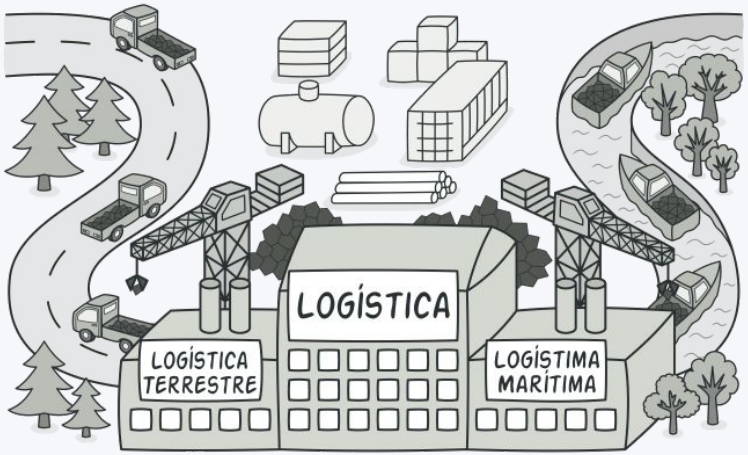
- Crear objetos de una superclase
- Redefiniendo el tipo exacto a crear

Se utiliza en:

- Creación de objetos dependientes de la plataforma (Windows, Linux, Mac, etc.)
- Pruebas y simulaciones (creación de mockups)

Solución:

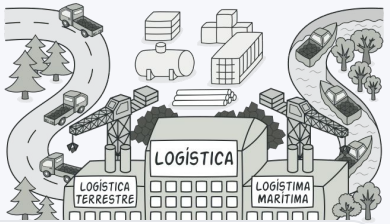
- La “Fábrica” hereda de una fábrica abstracta que crea “Productos”
- Todos los productos heredan de un producto abstracto con la funcionalidad mínima
- Los objetos no se crean con `new` sino llamando a un método “factory” específico
- Las Fábricas concretas crean y devuelven Productos concretos



CONCEPTO CLAVE

Encapsula la lógica de instanciación, independizando el código del cliente de las clases concretas que deben crearse.

Factory




```python
from abc import ABC, abstractmethod

class Factory(ABC):
    @abstractmethod
    def factory_method(self):
        pass

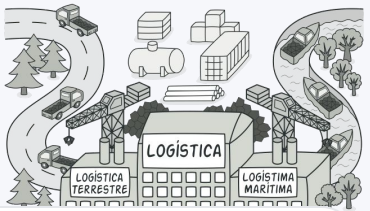
class ConcreteFactory(Factory):
    def factory_method(self):
        return ConcreteProduct()
```

```python
class Product(ABC):
    @abstractmethod
    def operation(self):
        pass

class ConcreteProduct(Product):
    def operation(self):
        return "Product operation"

```

Factory: Ejemplo

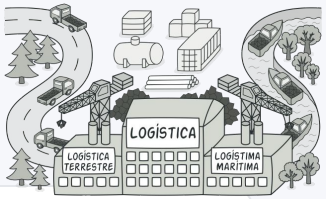



```python
from factory import Factory, Product
from random import choice, randint, random

class Veterinaria (Factory):
    def factory_method(self):
        nombres = ("Boby","Batuque","Sultan","Michifus","Rosalía")
        return Mascota(choice(nombres),randint(1,10),random()*10000)

class Forrajeria (Factory):
    def factory_method(self):
        nombres = ("Royal Canin","KMeal","Wiskas","Suelto","Ravioles")
        return Alimento(choice(nombres),randint(2,10),random()*8000)
```

Factory: Ejemplo



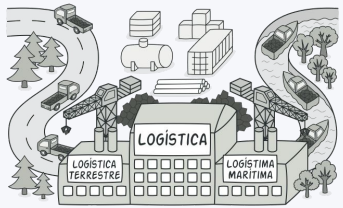

```python
class Mascota(Product):
    def __init__(self, nombre : str, cantidad : int, precio : float):
        self.nombre = nombre
        self.cantidad = cantidad
        self.precio = precio
    def __str__(self):
        return f"Mascota: {self.nombre}, Cantidad: {self.cantidad}, Precio: {self.precio}"
    def operation (self):
        return self.cantidad*self.precio

```

```python
class Alimento(Product):
    def __init__(self, nombre : str, cantidad : int, precio : float):
        self.nombre = nombre
        self.cantidad = cantidad
        self.precio = precio
    def __str__(self):
        return f"Alimento: {self.nombre}, Cantidad: {self.cantidad}, Precio: {self.precio:.2f}"
    def operation (self):
        return self.cantidad*self.precio

```

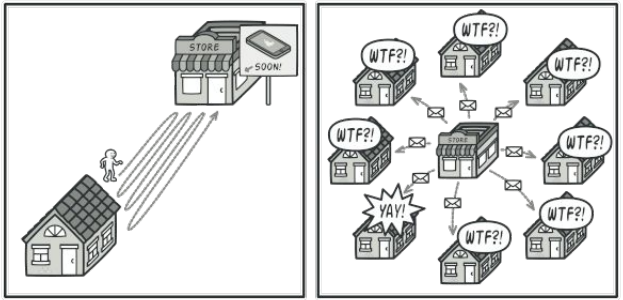
Factory: Ejemplo




```python
proveedores = [Veterinaria(),Forrajeria()]
productos = []
for i in range (10):
    producto = choice(proveedores).factory_method ()
    productos.append(producto)
for producto in productos:
    print (producto)
    print (f"TOTAL: {producto.operation():.2f}")

```

Patrón Observer



01 Problema

Problema que resuelve:

- Notificarse de los cambios en un objeto
- Sin heredar y sin acoplamiento

02 Casos de Uso

Se utiliza en:

- Interfaces de usuario
- Monitoreo

03 Solución

Mecanismo Observer:

- El “observador” implementa una interfaz común para ser notificado de cambios
- El “observador” se “suscribe” al servidor
- El servidor notifica cambios a todos los observadores suscriptos

Patrón Observer

```python
"""Ejemplos de aplicacion del patron Observer
"""
from abc import ABC,abstractmethod

class Subject:
    def __init__(self):
        self._observers = []

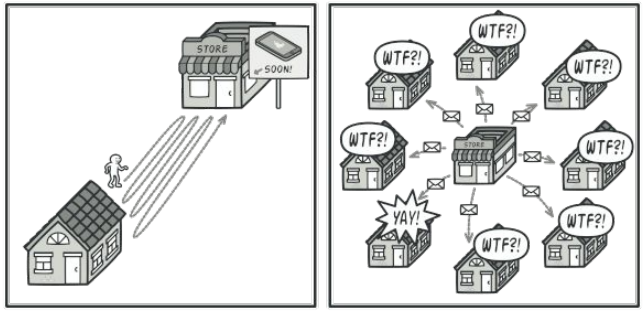
    def attach(self, observer):
        self._observers.append(observer)

    def notify(self):
        for observer in self._observers:
            observer.update()

class Observer(ABC):
    @abstractmethod
    def update(self):
        """
        Todas las clases derivadas deben definir este
        método, para poder recibir notificaciones de actualización
        """
        pass

class ConcreteObserver(Observer):
    def update(self):
        print("ConcreteObserver received an update")

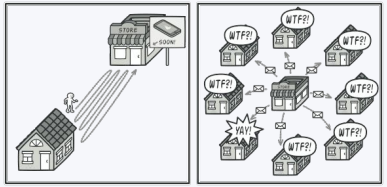

```



Conceptos clave

- **Subject:** Mantiene la lista de observadores y notifica cambios.
- **Observer:** Interfaz para recibir actualizaciones.
- **Desacoplamiento:** Relación 1-a-N sin dependencia rígida.

Observer: Ejemplo




```python
from observer import Subject, Observer

class ListaPrecios (Subject):
    def __init__ (self):
        super().__init__()

class Cliente (Observer):
    def __init__ (self,nombre : str):
        super().__init__()
        self.nombre = nombre

    def update(self):
        print (f"Nombre: {self.nombre}. "
               "LISTA DE PRECIOS ACTUALIZADA.")

```

```python
# Ahora utilizamos el Sujeto
# y el Observador
lista = ListaPrecios()
cliente1 = Cliente("Martin")
cliente2 = Cliente("Ivan")
cliente3 = Cliente("Analia")

lista.attach(cliente1)
lista.attach(cliente2)
lista.attach(cliente3)

lista.notify()
```

DAO

¡Gracias!

Hasta la próxima

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA